



Security Review For Incentiv Network



Collaborative Audit Prepared For:
Lead Security Expert(s):

Incentiv Network

Oxadrii

Oxdeadbeef

Date Audited:

September 9 - September 14, 2025

Introduction

Incentiv is a Layer 1 blockchain built to simplify and democratize blockchain technology, making it accessible, intuitive, and rewarding for users of all backgrounds. Its purpose is to bridge the gap between Web2 and Web3, offering a seamless user experience that empowers both developers and non-technical users to engage with the blockchain effortlessly.

Scope

Repository: IncentivNetwork/core-contracts

Audited Commit: c2be58205780ff4c693e73cf85aee7325d1c27c1

Final Commit: d55b3ab1989e74378b2035d82ab1c6819381024a

Files:

- contracts/core/SenderCreator.sol
- contracts/incentiv/BaseIncentivAccount.sol
- contracts/incentiv/Faucet.sol
- contracts/incentiv/IncentivAccountFactory.sol
- contracts/incentiv/IncentivAccount.sol
- contracts/incentiv/RewardDistribution.sol
- contracts/incentiv/TransferGate.sol
- contracts/incentiv/UnifiedTokenPaymaster.sol
- contracts/interfaces/IIncentivAccountFactory.sol
- contracts/test/TestRewardDistribution.sol
- contracts/test/TestTransferGate.sol
- contracts/test/TestUnifiedTokenPaymaster.sol
- deploy/1_deploy_EntryPoint.ts
- deploy/2_deploy_IncentivAccountFactory.ts
- deploy/5_deploy_UnifiedTokenPaymaster.ts
- .github/workflows/build.yml
- .gitignore
- hardhat.config.ts
- package.json
- scripts/docker-gascal

- scripts/docker-gascalc.yml
- test/incentiv/baselIncentivAccount.test.ts
- test/incentiv/bulkBalanceReader.test.ts
- test/incentiv/contractRegistry.test.ts
- test/incentiv/faucet.test.ts
- test/incentiv/incentivAccountFactory.test.ts
- test/incentiv/incentivAccount.test.ts
- test/incentiv/incentivContractFactory.test.ts
- test/incentiv/rewardDistribution.test.ts
- test/incentiv/transferGate.test.ts
- test/incentiv/unifiedTokenPaymaster.test.ts
- test/paymaster.test.ts
- test/y.bls.test.ts

Repository: IncentivNetwork/incentum-pow

Audited Commit: d85ae7555ec6070ec3e45e04338ebb27e7ecb04c

Final Commit: 074fbd8c71a128ab7b22ba4e557563b2fd890b71

Files:

- cmd/evm/internal/t8ntool/execution.go
- cmd/evm/internal/t8ntool/transition.go
- cmd/geth/main.go
- cmd/utls/flags.go
- consensus/beacon/consensus.go
- consensus/clique/clique.go
- consensus/ethash/consensus.go
- consensus/ethash/difficulty.go
- contracts/checkpointoracle/oracle.go
- core/chain_makers.go
- core/evm.go
- core/genesis.go
- core/rawdb/accessors_chain.go

- core/state_transition.go
- core/txpool/txpool.go
- core/vm/contracts.go
- core/vm/instructions.go
- core/vm/interpreter.go
- core/vm/jump_table_export.go
- core/vm/jump_table.go
- crypto/crypto.go
- crypto/secp256r1/verifier.go
- eth/catalyst/api.go
- eth/ethconfig/config.go
- internal/ethapi/api.go
- internal/ethapi/transaction_args_test.go
- les/catalyst/api_test.go
- p2p/peer.go
- params/config.go
- params/protocol_params.go
- tests/init.go

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
2	5	1

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Rewards derived from baseFee are incorrectly computed

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/63>

Summary

minerBaseFee and feePoolBaseFee are incorrectly computed, leading to a loss of rewards.

Vulnerability Detail

In `state_transition.go`, the `TransitionDb` function will compute the corresponding fees for the fee pool and the miner and add them as balance to Coinbase and the fee pool contract:

```
func (st *StateTransition) TransitionDb() (*ExecutionResult, error) {  
    ...SNIP  
} else {  
    gasUsed := new(big.Int).SetUint64(st.gasUsed())  
  
    // Check if fee pool is active  
    if st.evm.ChainConfig().IsFeePool(st.evm.Context.BlockNumber) {  
        tipFee := new(big.Int).Mul(gasUsed, effectiveTip)  
  
        // Split tip fee: 12.5% to miner, 87.5% to fee pool  
        minerTipFee := new(big.Int).Mul(tipFee,  
            ↳ big.NewInt(params.MinerFeePercent))  
        minerTipFee.Div(minerTipFee, big.NewInt(params.FeePercentDivisor))  
  
        feePoolTipFee := new(big.Int).Mul(tipFee,  
            ↳ big.NewInt(params.FeePoolPercent))  
        feePoolTipFee.Div(feePoolTipFee, big.NewInt(params.FeePercentDivisor))  
  
        if rules.IsLondon && st.evm.Context.BaseFee != nil {  
            minerBaseFee := new(big.Int).Mul(st.evm.Context.BaseFee,  
                ↳ big.NewInt(params.MinerFeePercent)) <@  
            minerBaseFee.Div(minerBaseFee, big.NewInt(params.FeePercentDivisor))  
  
            feePoolBaseFee := new(big.Int).Mul(st.evm.Context.BaseFee,  
                ↳ big.NewInt(params.FeePoolPercent))  
            feePoolBaseFee.Div(feePoolBaseFee,  
                ↳ big.NewInt(params.FeePercentDivisor))  
  
            // Add base fees to respective accounts
```

```

        st.state.AddBalance(st.evm.Context.Coinbase, minerBaseFee)
        st.state.AddBalance(params.FeePoolContractAddress, feePoolBaseFee)
    }

    st.state.AddBalance(st.evm.Context.Coinbase, minerTipFee)
    st.state.AddBalance(params.FeePoolContractAddress, feePoolTipFee)
} else {

```

The formula to compute the fee is given by $\text{fee} = \text{gasUsed} * \text{GasPrice}$. However, `minerBaseFee` and `feePoolBaseFee` are incorrectly computed. As shown in the snippet, the results are obtained by directly applying percentages to the `BaseFee`. This is incorrect, as `BaseFee` is actually a price, so the code misses multiplying by the actual `gasUsed` (just like it's done to derive `tipFee`).

Impact

High. Currently, the code will underestimate `minerBaseFee` and `feePoolBaseFee`, leading to a loss of rewards for `Coinbase` and the fee pool.

Code Snippet

[state_transition.go#L409](#)

Tool Used

Manual Review

Recommendation

Consider multiplying `st.evm.Context.BaseFee` by the gas used, and then apply the corresponding percentages (`MinerFeePercent` and `FeePoolPercent`) to the result of the product, instead of `BaseFee`.

Issue H-2: Malicious users can leverage addPending to bypass ETH transfer restrictions imposed by accounts

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/65>

Summary

Users can interact directly with addPending in the TransferGate to transfer ETH even if they're restricted by the target.

Vulnerability Detail

When a pending transaction is approved in the TransferGate, the logic to derive the address that msg.sender will be checked against is given by:

```
function approve(
    uint256 pendingId
)
    external
    nonReentrant
{
    ...SNIP
    require(
        msg.sender == (pending.tokenAddress != address(0) ?
            ↪ pending.tokenRecipient : pending.target),
        "Not authorized"
    );
    ...SNIP
}
```

As we can see, if pending.tokenAddress != address(0) then msg.sender will be verified against pending.tokenRecipient.

On the other hand, addPending allows anyone to directly add a pending transaction:

```
function addPending(
    address target,
    address tokenAddress,
    uint256 tokenAmount,
    address tokenRecipient
)
    external
    payable
```

```

{
    uint256 pendingId = nextPendingId++;

    pendingTransactions[pendingId] = PendingTransaction({
        sender: msg.sender,
        target: target,
        value: msg.value,
        tokenAddress: tokenAddress,
        tokenAmount: tokenAmount,
        tokenRecipient: tokenRecipient,
        isProcessed: false
    });

    if (tokenAddress != address(0) && tokenAmount > 0) {
        IERC20(tokenAddress).safeTransferFrom(msg.sender, address(this),
        ↪ tokenAmount);
    }

    emit PendingTransactionCreated(
        pendingId,
        msg.sender,
        target,
        msg.value,
        tokenAddress,
        tokenAmount,
        tokenRecipient
    );
}

```

As shown, there's no restrictions to whether users can send both ETH and tokens at the same time. An attacker can then leverage this to transfer ETH to an account, even if the account has blacklisted them, by performing the following attack:

1. An attacker A wants to send ETH to address B. Address B has blacklisted any txn incoming from address A.
2. The attacker calls `addPending` with a fake `tokenAddress`, and sets themselves as `tokenRecipient`. He sets `target` to address B, and passes some value.
3. The attacker then calls `approve`. Because of the check in line 350, `msg.sender` is compared with `pending.tokenRecipient`, which is the attacker themselves, so the check passes.
4. Finally, the ETH is transferred from the gate to the target, effectively bypassing any custom restriction

Impact

High. Account restrictions can be bypassed by leveraging `addPending`, breaking the expected core behavior of the transfer gate.

Code Snippet

[TransferGate.sol#L307](#)

Tool Used

Manual Review

Recommendation

Consider adding a check in `addPending` so that only an ETH transfer or a token transfer is allowed to be done at the same time. In case the caller wants to transfer both, the transaction should revert.

Discussion

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/32>

Issue M-1: Global blacklist should be checked on from

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/64>

Summary

In `checkApproval`, the `globalBlacklist` is never checked on `from`, allowing them to bypass blacklists by transferring tokens via `transferFrom` instead of a regular transfer.

Vulnerability Detail

In `checkApproval`, the code correctly checks all `accountConfig` configured flags against the `from` address. However, the `globalBlacklist` is never checked for `from`, and is only checked for `msg.sender`.

```
function checkApproval(
    address target,
    uint256 value,
    bytes calldata data
)
    external
    view
    returns (ApprovalResult memory result)
{
    // If either the target or the sender is blacklisted, the transaction is
    ↪ rejected
    if (globalBlacklist[target] || globalBlacklist[msg.sender]) { <@ Never
    ↪ checked for `from`
        result.reject = RejectionReason.GLOBAL_BLACKLIST;
        return result;
    }
```

This allows a malicious user who is globally blacklisted to transfer tokens by approving a non-blacklisted account who acts as an intermediary, transferring tokens via `transferFrom`, instead of `transfer`.

Impact

Medium. Blacklisted users can bypass token transfer restrictions by leveraging `transferFrom`.

Code Snippet

[TransferGate.sol#L153](#)

Tool Used

Manual Review

Recommendation

Consider checking wether `from` is blacklisted.

Discussion

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/31>

Issue M-2: `transferFrom` transactions could fail if operator doesn't have enough funds

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/66>

Summary

The operator needs to pay `transferFrom` transactions, which is incorrect conceptually, leading to unexpected reverts.

Vulnerability Detail

When transferring assets via `transferFrom`, such transaction could end up in a pending state. The problem is that `addPending` will fetch the corresponding tokens from `msg.sender`, instead of the `from` address (who is the actual token holder). Because of this, the following scenario can occur:

1. Account A holds 10 tokens. They approve account B as operator, and account B triggers a `transferFrom` from A to C.
2. Account C has not whitelisted account A, making the transaction be pending.
3. When `addPending` is called, funds will be **pulled from account B, instead of account A, who is the actual token holder**. This is incorrect as funds should be taken from account A.

Impact

Medium. In the best scenario, the transaction will revert if the operator doesn't have enough funds to cover the transfer. In the worst scenario, assets will be incorrectly taken from the operator instead of `from`, which is conceptually incorrect.

Code Snippet

[TransferGate.sol#L318](#)

Tool Used

Manual Review

Recommendation

When `transferFrom` is triggered and the transaction is set in a pending state, tokens should be pulled from `from`, instead of `msg.sender`. Note that this will require `from` to have approved the `TransferGate`.

Discussion

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/49>

Issue M-3: Missing Storage Gaps

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/68>

Summary

Upgradeable contracts lack reserved storage gaps, increasing the risk of storage layout collisions in future upgrades. This can corrupt critical state (e.g., owner, recoveryAddress, registry pointers).

Vulnerability Detail

- Contracts marked upgradeable must preserve storage layout across versions. When new variables are inserted in base classes or reordering occurs, storage slots can shift.
- Without a reserved `__gap`, adding fields in inherited/base contracts later can overwrite existing state variables in derived contracts.
- This codebase's upgradeable contracts (e.g., `BaseIncentivAccount`, `IncentivAccount`) define storage but do not include `__gap`, risking collisions.

Additionally, imports are not to proxy supported contracts.

Impact

`BaseIncentivAccount` stores owner, publicKey, recoveryAddress, and the `ContractRegistry` reference. Corrupting any of these through an upgrade collision can:

1. Nullify or misdirect ownership checks
2. Corrupt `contractRegistry()` → redirect `entryPoint()`;

Code Snippet

Tool Used

Manual Review

Recommendation

Follow OZ recommendations on how to use storage gaps

https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage_gaps. Make sure its applied to all upgradable contracts and make sure upgradable contracts inherit contracts that support storage gaps.

Discussion

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/33>

Issue M-4: Batch execute not atomic

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/69>

Summary

executeBatch is non-atomic: when a subcall is marked PENDING or REJECTED by TransferGate, _call returns early for that item but the loop continues, executing subsequent operations.

Vulnerability Detail

BaseIncentivAccount.executeBatch loops and calls _call per item. It does not abort the batch on non-APPROVED items.

```
function executeBatch(address[] calldata dest, uint256[] calldata value, bytes[]
↪ calldata func) external {
    _requireFromEntryPointOrOwner();
    require(dest.length == func.length && (value.length == 0 || value.length ==
↪ func.length), "wrong array lengths");
    if (value.length == 0) {
        for (uint256 i = 0; i < dest.length; i++) {
            _call(dest[i], 0, func[i]);
        }
    } else {
        for (uint256 i = 0; i < dest.length; i++) {
            _call(dest[i], value[i], func[i]);
        }
    }
}
```

In IncentivAccount._call, when TransferGate returns PENDING or REJECTED the code returns only from that subcall, allowing the batch loop to continue.

```
if (result.reject == ITransferGate.RejectionReason.PENDING) {
    // If pending, add to pending transactions
    if (result.tokenAddress != address(0)) {
        IERC20 token = IERC20(result.tokenAddress);
        token.safeIncreaseAllowance(address(_tg), result.tokenAmount);
    }
    _tg.addPending{value: value}(target, result.tokenAddress, result.tokenAmount,
↪ result.tokenRecipient);
    _handleFee(target, value, data, 0);
    return;
} else if (result.reject != ITransferGate.RejectionReason.APPROVED) {
```

```
_tg.addRejected(target, value, result.tokenAddress, result.tokenAmount,  
    ↪ result.tokenRecipient, uint256(result.reject));  
return;  
}
```

Since the loop continues, later operations may execute even though earlier ones are pending (or later rejected)

Impact

Broken atomicity: batches can partially execute, violating common caller expectations for all-or-nothing. If transactions order are important, user can lose funds if one of the transactions fail.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider first checking the batch through TransferGate and not executing it if there are any pending or rejected transactions.

Discussion

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/34>

Issue M-5: No refund for fees on cancel/reject of pending transactions

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/70>

Summary

Fees are charged when a transfer is marked PENDING by TransferGate, but if that pending transaction is later rejected or canceled, there is no mechanism to refund the fee. Users can be charged for transactions that never execute.

Vulnerability Detail

In `IncentivAccount._call`, when `TransferGate.checkApproval` returns PENDING, the code calls `_handleFee(...)`, which enforces native/token fee payments.

If the pending transaction is later rejected (`TransferGate.reject`) or cancelled (`TransferGate.cancel`), the escrowed value/tokens are returned to the sender, but any fees already paid remain with `RewardDistribution`. There is no refund path for the fees.

Impact

Users incur fees for non-executed transactions (pending items that are later rejected/cancelled).

Code Snippet

Tool Used

Manual Review

Recommendation

Consider:

1. Charge on approval - only send the fees for reward distribution when the transaction is approved.
2. Add a refund-on-reject/cancel mechanism

Discussion

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/36>

Issue L-1: General improvements

Source:

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/issues/67>

Summary

Some code improvements were identified to increase code quality and readability.

Vulnerability Detail

The improvements include:

- In IncentivAccountFactory.sol, remove the checks performed in `createAccount`, given that they're already performed in `getAddress()`.
- The immutability comment is no longer valid, as the previous audit led to a change in how the Entry Point contract is stored (it is now fetched from the central registry, contrary to being immutably stored).
- UUPS references should be removed, as the code was migrated to use a beacon proxy pattern, instead of UUPS proxy pattern.
- The following substraction is incorrect. The input parsed until line 1114 goes from `[0..36]` (36 exclusive), having parsed a total of 36 bytes. Because of this, a 36 should be subtracted to `len(input)`, instead of a 37.

Impact

Informational/Code Improvement

Discussion

zpsmcdev

The following substraction is incorrect. The input parsed until line 1114 goes from `[0..36]` (36 exclusive), having parsed a total of 36 bytes. Because of this, a 36 should be subtracted to `len(input)`, instead of a 37.

The 37 is actually correct in this check.

<https://github.com/sherlock-audit/2025-09-incentiv-update-sept-5th/pull/47>
[allowbreak #discussion_r2368417445](#)

zpsmcdev

<https://github.com/IncentivNetwork/core-contracts/pull/35>

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.